Working with Files in Python3

Working with Files in Python

- Working with files allows programs to store, retrieve, and manage data from disk.
- Python provides built-in modules for handling files and folders efficiently.
- File operations are essential for automation, data processing, and reporting.
- Most file operations are performed using Python's standard library.
- Working with files is a common requirement in Data Science projects.
- Python supports text files, CSV files, JSON files, XML files, and archives.

Finding Files

- Finding files helps locate specific files within directories.
- Python can search files based on names and patterns.
- File searching is useful for automation workflows.
- Directory scanning helps organize large file systems.
- Pattern matching can filter required files.
- File discovery is often the first step in file processing.

Example

```
import os

for file in os.listdir():
  print(file)
```

Listing Directories

- Directories contain files and subdirectories.

- Python can display all contents of a directory.
- Directory listing helps inspect file structures.
- It is useful when automating file management tasks.
- Results can be filtered using conditions.
- The os module provides directory-related functions.

Example

```
import os

files = os.listdir("Documents")
print(files)
```

String Methods for File Processing

- String methods help manipulate file names and paths.
- They can check file extensions.
- Strings can be searched and modified easily.
- File names are often processed as strings.
- String methods simplify filtering operations.
- They are useful when handling large numbers of files.

Example

```
filename = "report.csv"

print(filename.endswith(".csv"))
```

Output

True

Pattern Matching with fnmatch

- fnmatch matches filenames using wildcard patterns.

- It supports Unix-style pattern matching.
- Wildcards help locate groups of files.
- It simplifies file filtering operations.
- Commonly used for automation scripts.
- It works across different operating systems.

Example

```
import fnmatch

fnmatch.fnmatch("report.csv", "*.csv")
```

Output

True

Advanced Pattern Matching

- Advanced matching combines multiple filtering rules.
- Patterns can target specific file types.
- Complex searches become easier to manage.
- Matching can be based on extensions or naming conventions.
- It improves automation efficiency.
- Large directory structures become easier to search.

Example

```
*.csv
report_*
data_2024_*
```

Pattern Matching with glob

- glob finds files matching specified patterns.
- It searches directories automatically.

- Recursive searches can be performed.
- Wildcards simplify searching multiple files.
- Results are returned as lists.
- It is widely used in Data Science projects.

Example

```
import glob

files = glob.glob("*.csv")
print(files)
```

Working with Files and Folders

- Files and folders organize data on storage devices.
- Python provides tools for managing both.
- Automation scripts frequently manipulate files.
- File operations improve workflow efficiency.
- Folder structures help organize projects.
- Python supports cross-platform file management.

Getting File Attributes

- File attributes provide information about files.
- Attributes include size, creation date, and modification date.
- Metadata helps track file changes.
- File information assists in auditing and reporting.
- Attributes can guide automation decisions.
- Python retrieves metadata through built-in modules.

Example

```
import os

size = os.path.getsize("report.csv")
print(size)
```

Traversing Directories

- Traversing explores folders and subfolders.
- It helps locate files in large directory trees.
- Recursive traversal automates searching.
- Directory traversal is useful for backups.
- It can process entire file systems efficiently.
- Python provides tools for recursive navigation.

Example

```
import os

for root, dirs, files in os.walk("."):
  print(files)
```

Copying Files

- Copying creates duplicates of existing files.
- Original files remain unchanged.
- Copies can be stored in different locations.
- Useful for backups and versioning.
- Python supports file duplication through modules.
- Automation scripts often use copying operations.

Example

```
import shutil

shutil.copy("report.txt", "backup/report.txt")
```

Moving Files

- Moving transfers files between locations.
- The original file is removed from its old location.
- Moving helps organize directories.
- Automation can sort files automatically.
- Moving is useful in workflow pipelines.
- Python provides built-in support for moving files.

Example

```
import shutil

shutil.move("report.txt", "archive/")
```

Renaming Files

- Renaming changes a file's name.
- It helps maintain naming consistency.
- Batch renaming improves organization.
- Automation can rename many files quickly.
- Renaming preserves file contents.
- Python provides simple renaming functions.

Example

```
import os

os.rename("old.txt", "new.txt")
```

Deleting Files

- Deleting removes files permanently.
- It helps free storage space.
- Care must be taken to avoid accidental loss.
- Automation can remove temporary files.
- Deletion is often used in cleanup scripts.
- Python provides functions for file removal.

Example

```
import os

os.remove("temp.txt")
```

Archiving Files

- Archives combine multiple files into one package.
- Compression reduces storage requirements.
- Archives simplify file sharing.
- ZIP is one of the most common archive formats.
- Archiving improves backup management.
- Python provides built-in ZIP support.

ZIP Files

- ZIP files store compressed collections of files.
- They save storage space.
- Multiple files can be grouped together.
- ZIP archives are widely supported.
- Python can create and extract ZIP files.

- ZIP files are commonly used for backups.

Example

archive.zip

Creating ZIP Files

- ZIP files can be created programmatically.
- Multiple files can be added to one archive.
- Compression reduces file sizes.
- Archives simplify file distribution.
- ZIP creation is useful in automation.
- Python provides the zipfile module.

Example

```
from zipfile import ZipFile

with ZipFile("archive.zip", "w") as zipf:
  zipf.write("report.txt")
```

Reading ZIP Files

- ZIP archives can be inspected without extraction.
- File names inside archives can be listed.
- Contents can be analyzed programmatically.
- Reading archives helps verify backups.
- Python allows direct access to archive metadata.
- ZIP reading does not modify the archive.

Example

```
from zipfile import ZipFile
```

```
with ZipFile("archive.zip") as zipf:
  print(zipf.namelist())
```

Extracting ZIP Files

- Extraction restores archived files.
- Files are copied from the archive to disk.
- Entire archives can be extracted at once.
- Extraction is useful for data imports.
- Python automates archive extraction.
- ZIP extraction preserves file contents.

Example

```
from zipfile import ZipFile

with ZipFile("archive.zip") as zipf:
  zipf.extractall("output")
```

Reading and Writing Files

- Reading retrieves data from files.
- Writing stores data into files.
- File operations support persistent storage.
- Most applications depend on file I/O.
- Python simplifies file access.
- File I/O is fundamental in Data Science.

Working with Text Files

- Text files store plain text information.

- They are human-readable.
- Text files are commonly used for logs and notes.
- Python can read and write text easily.
- Data can be processed line by line.
- Text files are lightweight and portable.

Example

```
with open("notes.txt", "r") as file:
  content = file.read()
```

Working with CSV Files

- CSV stands for Comma-Separated Values.
- CSV files store tabular data.
- They are widely used in Data Science.
- Each row represents a record.
- Python provides built-in CSV support.
- CSV files are compatible with spreadsheets.

Example

```
import csv

with open("data.csv") as file:
  reader = csv.reader(file)
```

Working with XML Files

- XML stores structured hierarchical data.
- Tags organize information.
- XML is common in configuration files.
- It supports nested structures.

- Python can parse XML documents.
- XML enables data exchange between systems.

Example

```
<student>
  <name>Sarah</name>
</student>
```

Working with JSON Files

- JSON stands for JavaScript Object Notation.
- JSON stores data as key-value pairs.
- It is widely used in APIs.
- JSON resembles Python dictionaries.
- Python easily converts JSON into objects.
- JSON is one of the most important data formats in Data Science.

Example

```
import json

with open("data.json") as file:
  data = json.load(file)
```

Persisting Objects

- Persistence saves object data for future use.
- Data can be stored on disk or databases.
- Objects can be restored later.
- Persistence prevents data loss.
- It enables long-term storage of program state.
- Machine learning applications often persist trained models.

Example

```
import pickle

with open("model.pkl", "wb") as file:
   pickle.dump(model, file)
```